Logística Express

Sistema de Gestión de Logística, Entregas y Rastreo

Proyecto BD NoSQL (MongoDB Atlas) · Opción 5

Materia: _____

Profesor(a): _____

Grupo: _____

Fecha de entrega: _____

Integrantes:

1. _____

2. _____

3. _____

4. _____

5. _____

Contenido

1. Introducción
2. Arquitectura del sistema (3 capas)
3. Modelo de datos en MongoDB
4. Reglas de validación e índice único
5. API REST (backend en Python)
6. Página web (frontend)
7. Flujo de datos: ejemplo completo
8. Instalación y ejecución
9. Pruebas realizadas en MongoDB Atlas
10. Cumplimiento de la rúbrica
11. Guion para la demostración
12. Preguntas que nos pueden hacer
13. Conclusiones
14. Anexo: código fuente

1. Introducción

Este proyecto es la **Opción 5: Sistema de Gestión de Logística, Entregas y Rastreo** (tipo UberEats / DHL). Lo llamamos **Logística Express**.

Hay dos tipos de usuario:

- **Cliente:** escribe su código de rastreo (`tracking_id`) y ve el estado de su paquete y por dónde ha pasado.
- **Administrador:** registra paquetes, los busca y filtra, agrega cada punto de control al historial, los marca como entregados y elimina órdenes que todavía no salen.

Objetivo

Construir un sistema web de 3 capas que guarde los datos en **MongoDB Atlas** (base de datos NoSQL en la nube), con una API en **Python (FastAPI + PyMongo)** y una página web que permita Crear, Consultar, Editar y Eliminar sin recargar.

¿Por qué MongoDB?

Un paquete cambia de estado muchas veces y va juntando eventos (puntos de control, imprevistos, entregas). En MongoDB esos eventos se guardan **dentro del mismo documento del paquete** en un arreglo llamado `historial_eventos`. Así, para ver todo el rastreo basta **una sola consulta**, sin `JOIN` ni tablas extra.

Además, no todos los paquetes tienen la misma información: algunos todavía no tienen repartidor o tienen diferentes etiquetas. MongoDB permite esa flexibilidad (esquema dinámico).

2. Arquitectura del sistema (3 capas)

El sistema sigue el esquema que pide el proyecto: **MongoDB Atlas ↔ API Python ↔ Página web**.

```
[ Página web ]      frontend/index.html + app.js
       ^
       |   JSON por HTTP (fetch)
       v
[ API Python ]      FastAPI: backend/main.py
       ^
       |   PyMongo (BSON)
       v
[ MongoDB Atlas ]   Cluster0 (M0) -> base logistica_db -> colección paquetes
```

Capa	Tecnología	Archivos
Datos (nube NoSQL)	MongoDB Atlas, clúster gratuito M0 `Cluster0`	En la nube: base `logistica_db`, colección `paquetes`
Backend (API)	Python + FastAPI + PyMongo	`backend/database.py`, `backend/main.py`, `backend/crear_bd.py`
Frontend (UI)	HTML + Bootstrap 5 + JavaScript (fetch)	`frontend/index.html`, `frontend/app.js`, `frontend/styles.css`

¿Qué hace cada archivo?

Archivo	Para qué sirve
`database.py`	Se conecta a Atlas con PyMongo. Lee la cadena de conexión del archivo `.env` (así la contraseña no se sube a GitHub).
`crear_bd.py`	Se corre una vez. Pone las reglas de validación, crea el índice único en `tracking_id` y carga 8 paquetes de ejemplo si la colección está vacía.
`main.py`	La API REST: un endpoint por cada operación (crear, consultar, editar, agregar evento, eliminar). También sirve la página web.
`index.html`	La página: pestaña de rastreo para el cliente y pestaña de administración.
`app.js`	Habla con la API con `fetch` y dibuja las tarjetas sin recargar la página.

3. Modelo de datos en MongoDB

Todo lo de un paquete vive en **un solo documento** de la colección `paquetes`. El documento tiene datos simples, subdocumentos y arreglos.

Ejemplo de documento (paquete PKG-100002)

```
{
  "_id": ObjectId("..."),
  "tracking_id": "PKG-100002",
  "estado": "En tránsito",
  "remitente": { "nombre": "Librería El Saber", "telefono": "5598765432",
                 "ciudad": "CDMX" },
  "destinatario": { "nombre": "Jorge Salinas", "telefono": "8187654321",
                    "direccion": "Calle Morelos 45, Centro",
                    "ciudad": "Monterrey" },
  "descripcion": "Caja con 6 libros",
  "peso_kg": 4.5,
  "repartidor": { "nombre": "Ana Torres", "vehiculo": "Moto" },
  "etiquetas": ["libros"],
  "historial_eventos": [
```

```
    { "fecha": ISODate("..."), "estado": "Creado",
      "ubicacion": "CDMX", "comentario": "Orden registrada" },
    { "fecha": ISODate("..."), "estado": "En tránsito",
      "ubicacion": "Querétaro", "comentario": "Punto de control Querétaro" }
  ],
  "total_eventos": 2,
  "fecha_creacion": ISODate("...")
}
```

Tipo de campo	Campos
Simple	`tracking_id`, `estado`, `descripcion`, `peso_kg`, `total_eventos`, `fecha_creacion`
Subdocumento (objeto dentro del documento)	`remitente`, `destinatario`, `repartidor`
Arreglo de textos	`etiquetas`
Arreglo de subdocumentos	`historial_eventos` (cada evento tiene fecha, estado, ubicación y comentario)

- `tracking_id` es el código de rastreo; tiene un **índice único**.
- `estado` es el estado actual del paquete (el del último evento).
- `repartidor` puede ser `null` si todavía no se asigna.
- `total_eventos` es un contador que sube con `$inc` cada vez que se agrega un evento.

Estados de un paquete

Creado → **En tránsito** → **En reparto** → **Entregado** (o **Cancelado**)

Datos de ejemplo

La colección tiene 8 paquetes de ejemplo, del `PKG-100001` al `PKG-100008`, con distintos estados, repartidores, etiquetas e historiales. Por ejemplo, el `PKG-100005` (televisión de 50 pulgadas) tiene un imprevisto "Retraso por clima" en su historial.

4. Reglas de validación e índice único

Aunque MongoDB es flexible, se pueden poner reglas mínimas para que no se guarden datos incompletos. Las reglas están en `backend/crear_bd.py` (variable `VALIDACION`) y usan `$jsonSchema`. Se aplican a la colección con el comando `collMod`.

MongoDB **rechaza** un documento si:

- Le falta alguno de: `tracking_id`, `estado`, `remitente`, `destinatario`, `descripcion`, `peso_kg`, `historial_eventos`, `total_eventos`.
- `estado` no es uno de los 5 estados permitidos.
- `peso_kg` es negativo o no es número.
- `remitente` no tiene `nombre` y `ciudad`, o `destinatario` no tiene `nombre`, `direccion` y `ciudad`.
- `etiquetas` no es un arreglo de textos.
- Algún evento del historial no tiene `fecha`, `estado` y `ubicacion`.

Índice único

Hay un índice **único** en `tracking_id`. Sirve para dos cosas: buscar rápido por código de rastreo y evitar que existan dos paquetes con el mismo código (MongoDB responde con el error `E11000 duplicate key`).

5. API REST (backend en Python)

La API está en `backend/main.py`. Cada función con `@app.get`, `@app.post`, `@app.put` o `@app.delete` es un endpoint. Recibe y responde JSON.

Método y ruta	Qué hace	PyMongo	Operadores
`POST /api/paquetes`	**C:** registra un paquete nuevo en estado Creado con su primer evento	`insert_one`	—
`GET /api/paquetes`	**R** (administración): lista con filtros `buscar`, `estado` y `repartidor`	`find` + proyección	`$regex`, `$or`, `$in`, `$slice`
`GET /api/rastreo/{tracking_id}`	**R** (cliente): estado e historial de un paquete	`find_one` + proyección	—
`GET /api/repartidores`	Nombres de repartidores para el filtro	`distinct`	—
`PUT /api/paquetes/{id}`	**U:** edita los datos del paquete	`update_one`	`$set`
`PUT /api/paquetes/{id}/evento`	**U:** agrega un punto de control y cambia el estado (también para marcar Entregado)	`update_one`	`$push`, `$set`, `$inc`, `$nin`
`DELETE /api/paquetes/{id}`	**D:** elimina el paquete **solo si está en Creado**	`delete_one`	—

El documento del proyecto usa `/api/recursos` como ejemplo; en nuestro sistema el recurso son los **paquetes**, por eso la ruta es `/api/paquetes`.

FastAPI genera sola una página para probar la API en **http://localhost:8000/docs**.

Operadores NoSQL usados (la rúbrica pide al menos 3)

Operador	Para qué lo usamos	Función
`$push`	Agrega un evento al final del arreglo `historial_eventos`	`agregar_evento`
`$set`	Cambia el estado general (por ejemplo a Entregado) y edita datos	`agregar_evento`, `editar_paquete`
`$inc`	Suma 1 a `total_eventos` cada vez que se agrega un evento	`agregar_evento`
`$in`	Filtra paquetes cuyo estado esté en la lista elegida	`listar_paquetes`
`$nin`	No deja agregar eventos a paquetes Entregado o Cancelado	`agregar_evento`

Operador	Para qué lo usamos	Función
`$regex`	Búsqueda por texto parcial, sin importar mayúsculas	`listar_paquetes`
`$or`	Busca el texto en varios campos a la vez	`listar_paquetes`
`$slice`	En la lista solo se manda el último evento del historial	`listar_paquetes`

Proyecciones (elegir qué campos se regresan)

- **Lista del administrador:** `{"historial_eventos": {"$slice": -1}}` → manda todo el paquete, pero del historial solo el último evento.
- **Rastreo del cliente:** solo `tracking_id`, `estado`, `descripcion`, nombre y ciudad del destinatario, nombre del repartidor e historial. **No** manda teléfonos ni dirección, porque el cliente no necesita verlos.

Reglas del negocio

- Un paquete **Entregado** o **Cancelado** ya no acepta eventos nuevos (filtro con `$nin`).
- Solo se pueden eliminar paquetes en estado **Creado**: si ya salió a ruta, borrarlo perdería su historial. El filtro del `delete_one` incluye `"estado": "Creado"`, así que MongoDB no borra nada si está en otro estado.

6. Página web (frontend)

Todo funciona **sin recargar la página**: `app.js` usa `fetch` para hablar con la API y vuelve a dibujar solo la parte que cambia.

Parte de la página	Qué hace	Llama a
Pestaña **Rastrear paquete** (cliente)	Caja para escribir el código y línea de tiempo con el historial	`GET /api/rastreo/{tracking_id}`
Pestaña **Administración**	Barra de búsqueda en tiempo real, filtro por repartidor y por estados, tarjetas de paquetes	`GET /api/paquetes`
Botón **Nuevo paquete** / **Editar**	Formulario dinámico: el repartidor aparece solo si se activa el interruptor; las etiquetas se agregan y quitan una por una	`POST` / `PUT /api/paquetes`
Botón **Evento**	Ventana para escribir el nuevo estado, la ubicación y un comentario	`PUT /api/paquetes/{id}/evento`
Botón **Entregado**	Agrega el evento Entregado en la ciudad del destinatario	`PUT /api/paquetes/{id}/evento`
Botón **Historial**	Muestra la línea de tiempo completa	`GET /api/rastreo/{tracking_id}`
Botón **Eliminar**	Solo aparece en paquetes en estado Creado	`DELETE /api/paquetes/{id}`

Los botones **Evento** y **Entregado** se ocultan cuando el paquete ya está Entregado o Cancelado.

Capturas del sistema

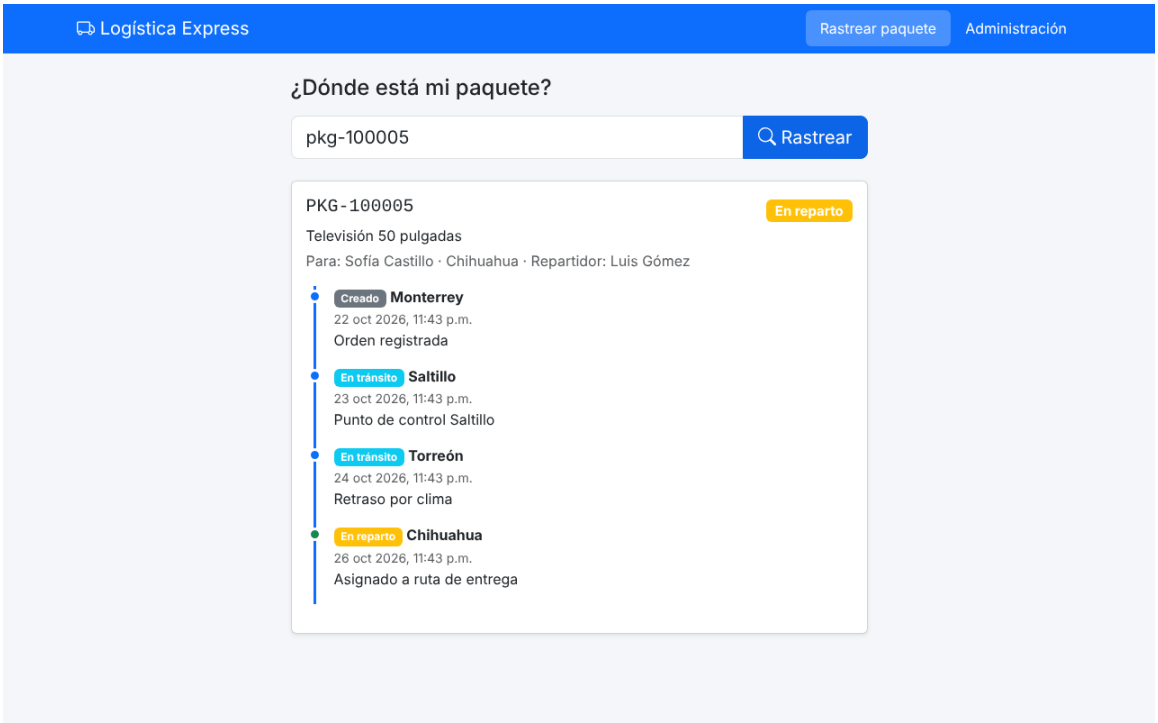


Figura 1. Rastreo del cliente con la línea de tiempo del paquete

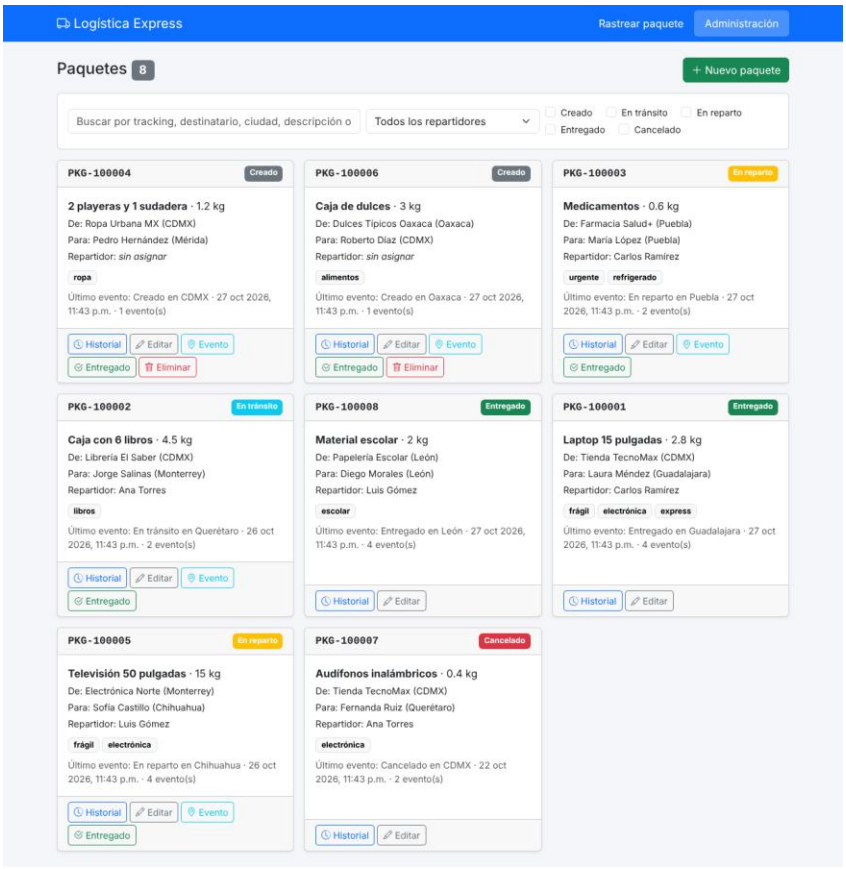


Figura 2. Módulo de administración con las tarjetas de paquetes

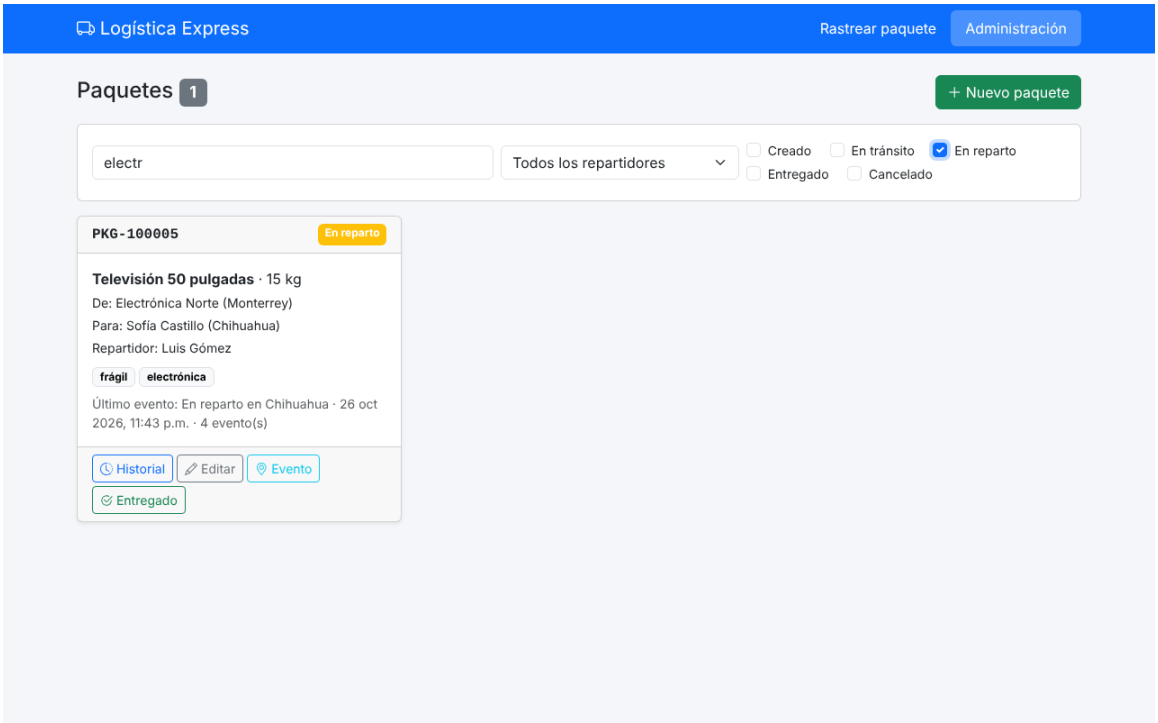


Figura 3. Búsqueda en tiempo real y filtros por estado y repartidor

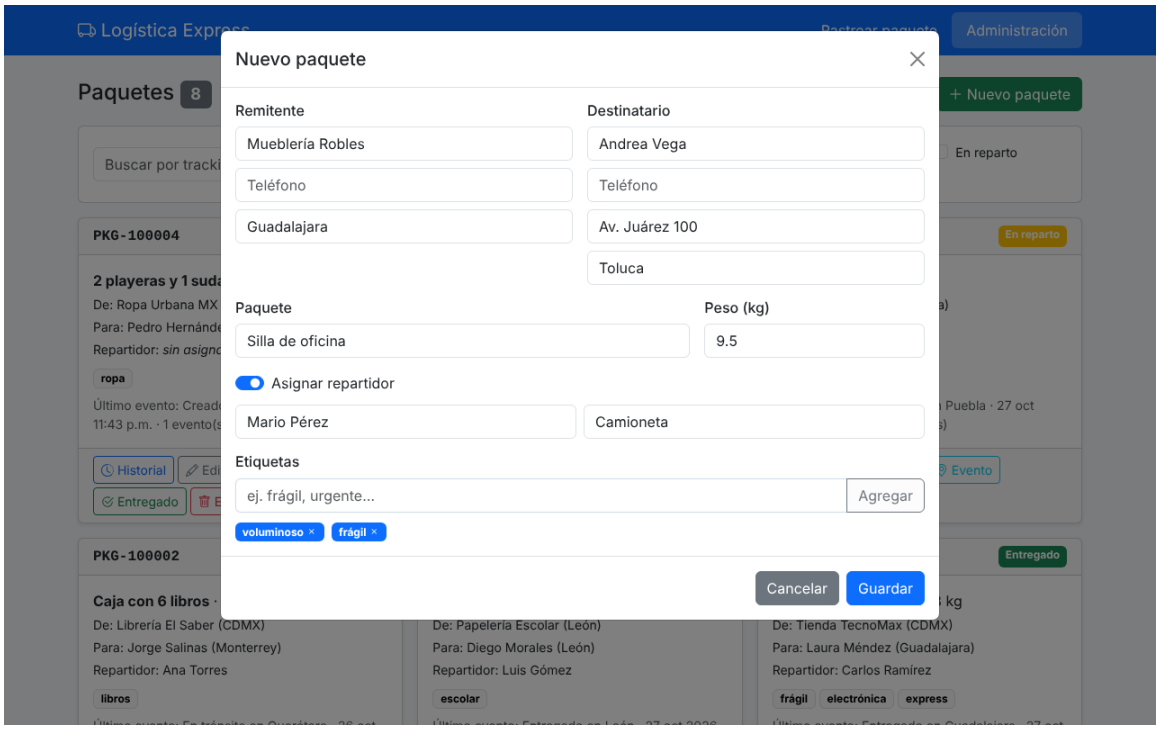


Figura 4. Formulario dinámico para crear un paquete

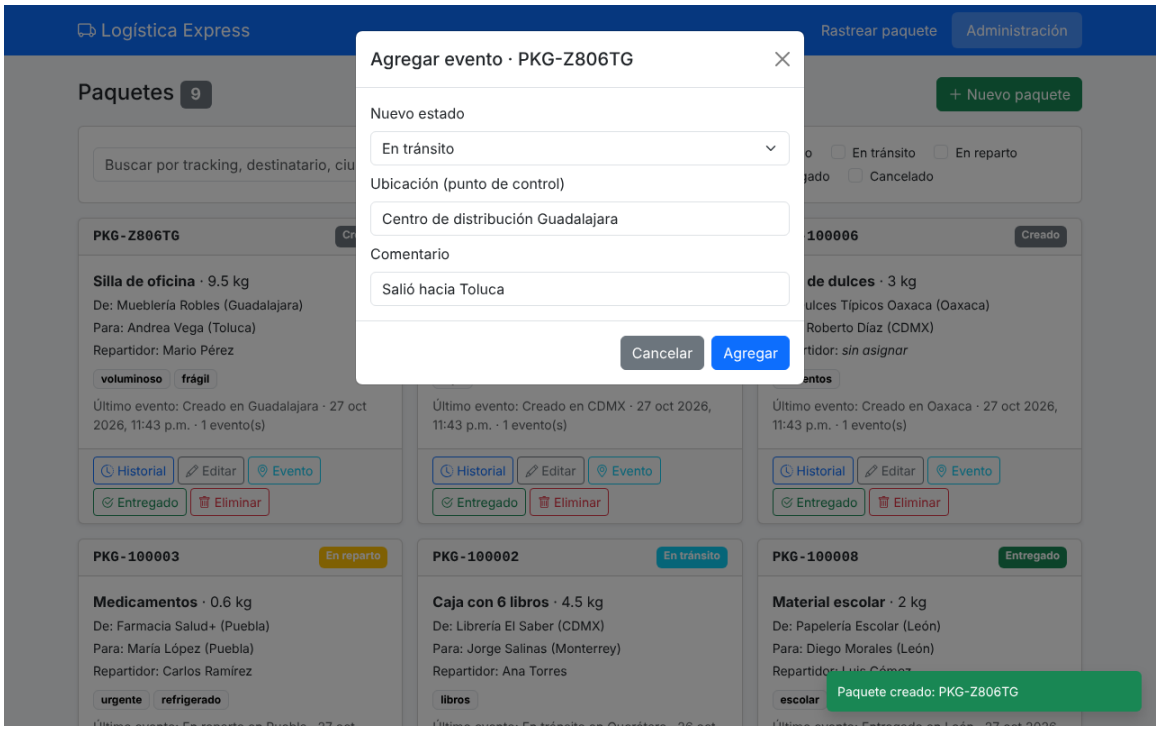


Figura 5. Ventana para agregar un evento (punto de control)

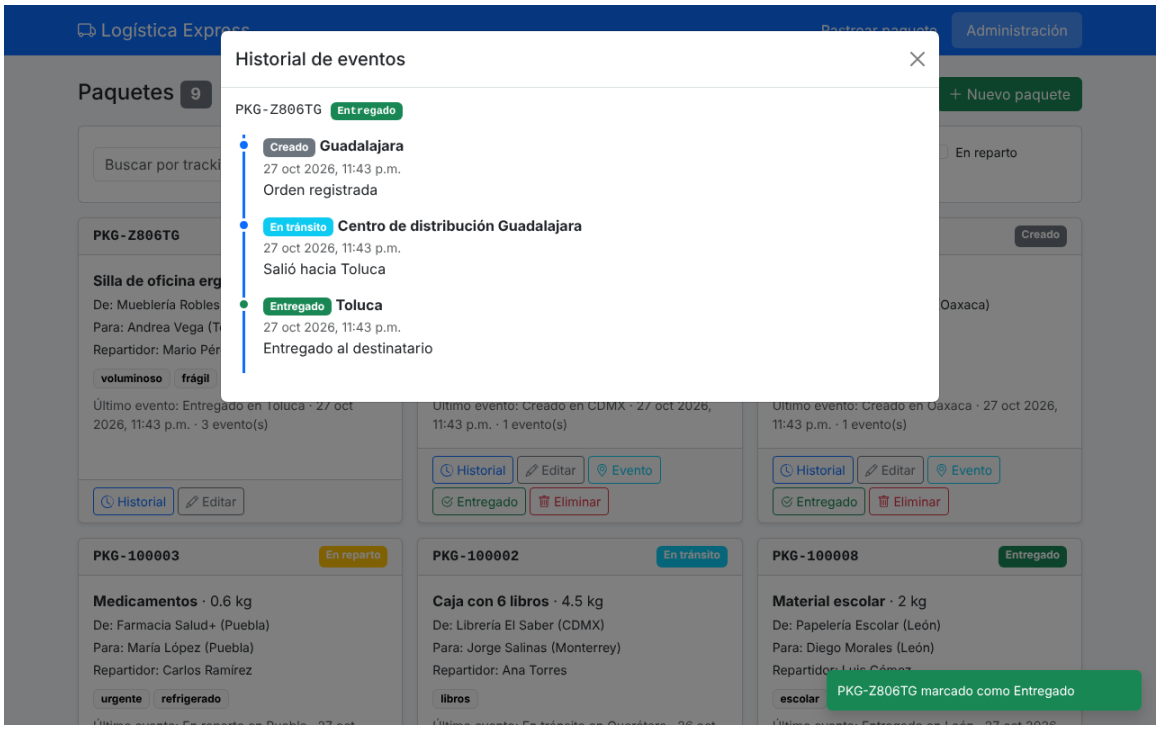


Figura 6. Historial completo de un paquete

7. Flujo de datos: ejemplo completo

Ejemplo: el administrador agrega un evento a un paquete.

1. En la página se llena la ventana **Evento** (estado "En reparto", ubicación "Monterrey") y se da clic en Agregar.
2. `app.js` manda `PUT /api/paquetes/<id>/evento` con el JSON `{"estado": "En reparto", "ubicacion": "Monterrey", "comentario": ""}`.
3. FastAPI revisa el JSON con el modelo `NuevoEvento` y ejecuta la función `agregar_evento` de `main.py`.
4. PyMongo manda la operación a Atlas en formato BSON. Atlas revisa las reglas de validación y guarda el cambio.
5. La API responde `{"mensaje": "Evento agregado: En reparto"}` y la página vuelve a pedir la lista. La tarjeta cambia sola.

La operación que se ejecuta en MongoDB

```
paquetes.update_one(
    {"_id": id, "estado": {"$nin": ["Entregado", "Cancelado"]}},   # filtro
    {"$push": {"historial_eventos": nuevo},     # agrega el evento al arreglo
     "$set":  {"estado": "En reparto"},         # cambia el estado general
```

```
    "$inc":  {"total_eventos": 1}})            # suma 1 al contador
```

En una sola operación se usan tres operadores: `$push`, `$set` y `$inc`. El `$nin` del filtro hace que no se modifiquen paquetes ya entregados o cancelados.

8. Instalación y ejecución

Requisitos: Python 3.10 o más nuevo y una cuenta de MongoDB Atlas con el clúster `Cluster0`.

En MongoDB Atlas

1. **Network Access:** agregar la IP de la computadora donde se va a correr ("Add Current IP Address"). Si se presenta en otra red (por ejemplo en la escuela), agregar esa IP ese día.
2. **Database Access:** usar un usuario con permiso para leer, escribir y administrar `logistica_db` (nosotros usamos `admin_user`).
3. **Cluster0 → Connect → Drivers → Python:** copiar la cadena de conexión.

En la computadora (PowerShell en Windows)

```
cd proyecto_mongodb_logistica
pip install -r requirements.txt
copy .env.example .env
# abrir .env y poner usuario y contraseña en MONGO_URI

cd backend
python crear_bd.py                 # validación + índice único + datos de ejemplo
python -m uvicorn main:app --reload   # levanta la API y la página
```

Después abrir **http://localhost:8000** (la página) o **http://localhost:8000/docs** (para probar la API sin la página).

Al correr `crear_bd.py` deben salir estos mensajes:

```
Validación actualizada en la colección 'paquetes'
Índice único en tracking_id listo
La colección ya tiene datos, no se insertó nada
```

Antes de presentar se puede dejar la base como al inicio con `python crear_bd.py reiniciar` (borra los paquetes y vuelve a cargar los 8 de ejemplo).

La contraseña

La cadena de conexión con la contraseña está en el archivo `.env`, que **no se sube a GitHub** porque está en `.gitignore`. En el repositorio solo está `.env.example` como plantilla.

9. Pruebas realizadas en MongoDB Atlas

El 28 de octubre de 2026 se corrió el sistema completo en una laptop con Windows, conectado al clúster real `Cluster0` de MongoDB Atlas. Estos fueron los resultados:

Preparación de la base

Prueba	**Resultado**
Correr `python crear_bd.py`	Correcto: validación actualizada, índice único listo y los 8 paquetes se conservaron
Validación activa en la colección	Correcto: activa en modo estricto (`strict`)
Índice `tracking_id_1`	Correcto: es único

Pruebas de la página y la API

Prueba	**Resultado**
Abrir http://localhost:8000	Correcto: la página carga
Rastrear `PKG-100005`	Correcto: Televisión 50 pulgadas, En reparto, 4 eventos
El rastreo no muestra teléfonos ni dirección	Correcto (proyección)
Buscar "frágil"	Correcto: 2 paquetes (`$regex` y `$or`)
Filtrar por estados Entregado y Cancelado	Correcto: 3 paquetes, todos con esos estados (`$in`)
Lista de repartidores	Correcto: Ana Torres, Carlos Ramírez, Luis Gómez (`distinct`)
Crear paquete	Correcto: se creó en estado Creado con su código de rastreo (`insert_one`)
Agregar evento En tránsito	Correcto: cambió el estado y creció el historial (`$push`, `$set`, `$inc`)
Marcar como Entregado	Correcto: estado Entregado con 3 eventos
Agregar evento a un paquete ya entregado	Rechazado, como debe ser: "El paquete no existe o ya fue entregado/cancelado" (`$nin`)
Eliminar un paquete entregado	Rechazado, como debe ser: "Solo se pueden eliminar paquetes en estado 'Creado'"
Eliminar un paquete en estado Creado	Correcto: se eliminó (`delete_one`)

Pruebas de las reglas de MongoDB

Prueba	Resultado
Insertar un documento sin `tracking_id`	Rechazado por MongoDB: "Document failed validation"
Insertar un paquete con estado inventado ("Perdido")	Rechazado por MongoDB (no está en la lista de estados)
Insertar un `tracking_id` repetido	Rechazado por MongoDB: error E11000 duplicate key

Los paquetes creados para las pruebas se borraron al terminar; la colección quedó con los 8 paquetes de ejemplo.

10. Cumplimiento de la rúbrica

Criterio	Valor	Cómo lo cumplimos
Modelado NoSQL y Atlas	25%	Clúster M0 en Atlas. Un documento por paquete con subdocumentos (remitente, destinatario, repartidor) y arreglos (etiquetas, historial de eventos). Validación con `$jsonSchema` e índice único. Secciones 3 y 4.
Capa de Datos (Python)	30%	API en FastAPI con PyMongo: `insert_one`, `find`, `find_one`, `distinct`, `update_one`, `delete_one`, conectada a Atlas. Sección 5 y anexo.
Interfaz Web (CRUD)	25%	Página con Bootstrap: crear, consultar, editar y eliminar sin recargar (fetch). Formularios dinámicos, tarjetas con botones y búsqueda en tiempo real. Sección 6.
Uso de operadores NoSQL	10%	Ocho operadores: `$push`, `$set`, `$inc`, `$in`, `$nin`, `$regex`, `$or`, `$slice`. Sección 5.
Demostración y reporte	10%	Este reporte, el README del repositorio, el código comentado en español y el guion de demostración de la sección 11.

Lo que pide la Opción 5

Requisito	Dónde está
C: registro de una nueva orden o paquete	`POST /api/paquetes` y botón Nuevo paquete
R: cliente consulta el estado por `tracking_id`	`GET /api/rastreo/{tracking_id}` y pestaña Rastrear paquete
R: administración filtra por estado o por repartidor	`GET /api/paquetes?estado=...&repartidor=...` y filtros de la pestaña Administración
U: agregar evento al historial (`$push`)	`PUT /api/paquetes/{id}/evento` y botón Evento
U: cambiar estado a Entregado (`$set`)	Mismo endpoint y botón Entregado
D: eliminar órdenes en estado Creado	`DELETE /api/paquetes/{id}` y botón Eliminar

11. Guion para la demostración (5 minutos)

1. **Atlas:** mostrar `Cluster0` → `logistica_db` → `paquetes`. Abrir un documento y señalar los subdocumentos y arreglos. En la pestaña Validation mostrar las reglas y en Indexes el índice único de `tracking_id`.

2. **Cliente:** en Rastrear paquete escribir `PKG-100005`. Se ve el historial, incluido el imprevisto "Retraso por clima".
3. **Crear:** Administración → Nuevo paquete. Llenar los datos, activar el repartidor y agregar etiquetas. El paquete aparece arriba en estado Creado. Refrescar Atlas para ver el documento nuevo.
4. **Buscar y filtrar:** escribir "electr" en la barra (filtra mientras se escribe), marcar el estado En reparto y elegir un repartidor.
5. **Evento (`$push`, `$set`, `$inc`):** al paquete nuevo darle Evento → En tránsito. Mostrar que `total_eventos` pasa de 1 a 2. Luego darle Entregado y mostrar el arreglo `historial_eventos` en Atlas.
6. **Eliminar:** eliminar un paquete en Creado. Explicar que los paquetes que ya salieron no tienen botón de eliminar y que la API también lo impide.

12. Preguntas que nos pueden hacer

¿Qué es un subdocumento?

Un objeto dentro del documento. Por ejemplo `destinatario`, que tiene nombre, teléfono, dirección y ciudad.

¿Por qué el historial va dentro del paquete y no en otra colección?

Porque siempre se consulta junto con el paquete. Así una sola consulta trae todo el rastreo, sin `JOIN`.

¿Qué hace $push?

Agrega un elemento al final de un arreglo. Lo usamos para cada evento nuevo del historial.

¿Qué hace $set?

Cambia el valor de un campo sin tocar los demás. Lo usamos para el estado y para editar.

¿Qué hace $inc?

Suma (o resta) a un número. Lo usamos para el contador `total_eventos`.

¿Qué hace $in?

Busca documentos cuyo campo tenga alguno de los valores de una lista (filtro de estados).

¿Qué hace $nin?

Lo contrario de `$in`: el valor NO debe estar en la lista. Lo usamos para no modificar paquetes entregados o cancelados.

¿Qué es una proyección?

Elegir qué campos regresa la consulta. Al cliente no le mandamos teléfonos ni dirección.

¿Para qué sirve la validación si MongoDB es "sin esquema"?

MongoDB es flexible, pero podemos poner reglas mínimas para que no se guarden datos incompletos o con estados inventados.

¿Para qué sirve el índice único?

Para buscar rápido por `tracking_id` y para que no haya dos paquetes con el mismo código.

¿Por qué solo se pueden borrar paquetes en Creado?

Porque si ya salió a ruta, borrarlo perdería su historial. El filtro del `delete_one` incluye `"estado": "Creado"`.

¿Qué es PyMongo?

La librería oficial de Python para conectarse a MongoDB.

¿Qué es FastAPI?

Un framework de Python para hacer APIs. Cada función con `@app.get`, `@app.post`, etc. es un endpoint.

¿Qué es BSON?

El formato en que MongoDB guarda y transmite los documentos. Es como JSON, pero binario y con más tipos (fechas, ObjectId).

¿Dónde está la contraseña?

En el archivo `.env`, que no se sube a GitHub (está en `.gitignore`).

¿Por qué no se recarga la página?

Porque JavaScript usa `fetch` para pedir y mandar datos a la API, y solo cambia la parte de la página que se necesita.

13. Conclusiones

- MongoDB se adapta muy bien a la logística: el historial de cada paquete se guarda dentro del mismo documento y se consulta completo en una sola operación.
- Con operadores como `$push`, `$set` e `$inc` se actualiza el paquete en una sola operación, sin leerlo primero.
- Las reglas de validación y el índice único dan orden a los datos sin perder la flexibilidad de NoSQL.
- Separar el sistema en 3 capas (Atlas, API en Python y página web) permite cambiar una parte sin afectar a las otras.

Anexo: código fuente

El código completo está en el repositorio de GitHub `Vizpok/Sistema_Inventario_Proyecto`, carpeta `proyecto_mongodb_logistica`. Aquí se incluye el backend y las partes principales de la página.

backend/database.py (conexión a Atlas)

```
# ------------------------------------------------------------
# Conexión a MongoDB Atlas con PyMongo
# ------------------------------------------------------------
# La cadena de conexión NO se escribe en el código: se lee del
# archivo .env (variable MONGO_URI) para no subir la contraseña a GitHub.

import os

from dotenv import load_dotenv
from pymongo import MongoClient

load_dotenv()  # lee el archivo .env

MONGO_URI = os.getenv("MONGO_URI")
NOMBRE_BD = os.getenv("MONGO_DB", "logistica_db")

if not MONGO_URI:
    raise RuntimeError("Falta MONGO_URI en el archivo .env (copia .env.example a .env)")

# MongoClient abre la conexión con el clúster de Atlas
cliente = MongoClient(MONGO_URI, tz_aware=True)  # tz_aware: fechas con zona horaria (UTC)

# Base de datos y colección que usa todo el sistema
bd = cliente[NOMBRE_BD]
paquetes = bd["paquetes"]

# Estados posibles de un paquete (los usa la validación y la API)
ESTADOS = ["Creado", "En tránsito", "En reparto", "Entregado", "Cancelado"]
```

backend/main.py (API REST)

```
# ------------------------------------------------------------
# API REST del sistema de logística (FastAPI + PyMongo)
# ------------------------------------------------------------
# Para correrla (desde la carpeta backend):
#     uvicorn main:app --reload
# Luego abrir http://localhost:8000  (página web)
#          o http://localhost:8000/docs (documentación automática de la API)

import random
import re
import string
from datetime import datetime, timezone
from pathlib import Path

from bson import ObjectId
```

```
from fastapi import FastAPI, HTTPException, Query
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field

from database import ESTADOS, paquetes

app = FastAPI(title="Sistema de Logística y Rastreo")



# ---------- Modelos: qué datos recibe la API ----------
class Remitente(BaseModel):
    nombre: str
    telefono: str = ""
    ciudad: str



class Destinatario(BaseModel):
    nombre: str
    telefono: str = ""
    direccion: str
    ciudad: str



class Repartidor(BaseModel):
    nombre: str
    vehiculo: str = ""



class DatosPaquete(BaseModel):
    """Datos que se capturan al crear o editar un paquete."""
    remitente: Remitente
    destinatario: Destinatario
    descripcion: str
    peso_kg: float = Field(gt=0)
    repartidor: Repartidor | None = None
    etiquetas: list[str] = []



class NuevoEvento(BaseModel):
    """Un punto de control: el paquete pasó por un lugar y cambió de estado."""
    estado: str
    ubicacion: str
    comentario: str = ""



# ---------- Funciones de apoyo ----------
def a_json(doc):
    """Convierte el _id (ObjectId) a texto para poder mandarlo como JSON."""
    if doc and "_id" in doc:
        doc["_id"] = str(doc["_id"])
    return doc



def id_valido(id):
    """Revisa que el id tenga formato de ObjectId; si no, responde 404."""
```

```
    if not ObjectId.is_valid(id):
        raise HTTPException(404, "Paquete no encontrado")
    return ObjectId(id)


def nuevo_tracking_id():
    """Genera un código de rastreo como PKG-4F7K2Q."""
    return "PKG-" + "".join(random.choices(string.ascii_uppercase + string.digits, k=6))


# ============================================================
# C - CREAR:  POST /api/paquetes   ->  insert_one
# ============================================================
@app.post("/api/paquetes", status_code=201)
def crear_paquete(datos: DatosPaquete):
    ahora = datetime.now(timezone.utc)
    documento = datos.model_dump()  # convierte el modelo a diccionario
    documento["tracking_id"] = nuevo_tracking_id()
    documento["estado"] = "Creado"
    documento["fecha_creacion"] = ahora
    # El historial empieza con un primer evento (arreglo de subdocumentos)
    documento["historial_eventos"] = [{
        "fecha": ahora,
        "estado": "Creado",
        "ubicacion": datos.remitente.ciudad,
        "comentario": "Orden registrada",
    }]
    documento["total_eventos"] = 1

    resultado = paquetes.insert_one(documento)
    return {"mensaje": "Paquete creado", "_id": str(resultado.inserted_id),
            "tracking_id": documento["tracking_id"]}


# ============================================================
# R - CONSULTAR (módulo de administración):
#     GET /api/paquetes?buscar=...&estado=...&repartidor=...  ->  find
# ============================================================
@app.get("/api/paquetes")
def listar_paquetes(
    buscar: str = "",
    estado: list[str] = Query(default=[]),
    repartidor: str = "",
):
    filtro = {}

    # Barra de búsqueda: busca el texto en varios campos con $regex ("contiene", sin
    # importar mayúsculas). $or = basta con que coincida en uno de los campos.
    if buscar:
        texto = {"$regex": re.escape(buscar), "$options": "i"}
        filtro["$or"] = [
            {"tracking_id": texto},
            {"destinatario.nombre": texto},   # campo dentro de un subdocumento
            {"destinatario.ciudad": texto},
            {"descripcion": texto},
            {"etiquetas": texto},             # busca dentro del arreglo
        ]
```

```
    # Filtro por estado: $in = el estado está dentro de la lista elegida
    if estado:
        filtro["estado"] = {"$in": estado}

    # Filtro por nombre del repartidor (subdocumento)
    if repartidor:
        filtro["repartidor.nombre"] = repartidor

    # Proyección: del historial solo mandamos el último evento ($slice: -1)
    proyeccion = {"historial_eventos": {"$slice": -1}}

    cursor = paquetes.find(filtro, proyeccion).sort("fecha_creacion", -1)
    return [a_json(doc) for doc in cursor]


# ============================================================
# R - CONSULTAR (módulo de cliente):
#     GET /api/rastreo/{tracking_id}  ->  find_one por tracking_id
# ============================================================
@app.get("/api/rastreo/{tracking_id}")
def rastrear(tracking_id: str):
    # Proyección: el cliente solo ve lo necesario (sin teléfonos ni dirección)
    proyeccion = {
        "_id": 0,
        "tracking_id": 1,
        "estado": 1,
        "descripcion": 1,
        "destinatario.nombre": 1,
        "destinatario.ciudad": 1,
        "repartidor.nombre": 1,
        "historial_eventos": 1,
    }
    doc = paquetes.find_one({"tracking_id": tracking_id.strip().upper()}, proyeccion)
    if not doc:
        raise HTTPException(404, "No existe un paquete con ese código de rastreo")
    return doc


# Lista de repartidores para el filtro del administrador (distinct = valores sin repetir)
@app.get("/api/repartidores")
def listar_repartidores():
    return sorted(n for n in paquetes.distinct("repartidor.nombre") if n)


# ============================================================
# U - ACTUALIZAR datos del paquete:
#     PUT /api/paquetes/{id}   ->  update_one con $set
# ============================================================
@app.put("/api/paquetes/{id}")
def editar_paquete(id: str, datos: DatosPaquete):
    resultado = paquetes.update_one(
        {"_id": id_valido(id)},
        {"$set": datos.model_dump()},  # $set reemplaza solo los campos indicados
    )
    if resultado.matched_count == 0:
        raise HTTPException(404, "Paquete no encontrado")
```

```
    return {"mensaje": "Paquete actualizado"}


# ============================================================
# U - AGREGAR EVENTO al historial (punto de control):
#     PUT /api/paquetes/{id}/evento  ->  update_one con $push, $set y $inc
# ============================================================
@app.put("/api/paquetes/{id}/evento")
def agregar_evento(id: str, evento: NuevoEvento):
    if evento.estado not in ESTADOS or evento.estado == "Creado":
        raise HTTPException(400, "Estado no válido")

    nuevo = {
        "fecha": datetime.now(timezone.utc),
        "estado": evento.estado,
        "ubicacion": evento.ubicacion,
        "comentario": evento.comentario,
    }
    resultado = paquetes.update_one(
        # Solo paquetes que sigan activos ($nin = "no está en la lista")
        {"_id": id_valido(id), "estado": {"$nin": ["Entregado", "Cancelado"]}},
        {
            "$push": {"historial_eventos": nuevo},  # agrega el evento al final del arreglo
            "$set": {"estado": evento.estado},       # cambia el estado general
            "$inc": {"total_eventos": 1},            # suma 1 al contador de eventos
        },
    )
    if resultado.matched_count == 0:
        raise HTTPException(400, "El paquete no existe o ya fue entregado/cancelado")
    return {"mensaje": f"Evento agregado: {evento.estado}"}


# ============================================================
# D - ELIMINAR:  DELETE /api/paquetes/{id}  ->  delete_one
#     Regla: solo se eliminan órdenes en estado "Creado" (aún no despachadas)
# ============================================================
@app.delete("/api/paquetes/{id}")
def eliminar_paquete(id: str):
    resultado = paquetes.delete_one({"_id": id_valido(id), "estado": "Creado"})
    if resultado.deleted_count == 0:
        raise HTTPException(400, "Solo se pueden eliminar paquetes en estado 'Creado'")
    return {"mensaje": "Paquete eliminado"}


# ---------- Página web (frontend) ----------
# Se sirve la carpeta frontend en la raíz: http://localhost:8000
CARPETA_FRONTEND = Path(__file__).resolve().parent.parent / "frontend"
app.mount("/", StaticFiles(directory=CARPETA_FRONTEND, html=True), name="frontend")
```

backend/crear_bd.py (validación, índice y datos de ejemplo)

```
# ------------------------------------------------------------
# Preparación de la base de datos en MongoDB Atlas
# ------------------------------------------------------------
# Se ejecuta UNA vez:   python crear_bd.py
# Para borrar todo y volver a cargar los datos de ejemplo:
#                       python crear_bd.py reiniciar
#
```

```
# Hace tres cosas:
#   1. Crea la colección "paquetes" con reglas de validación (Schema Validation).
#   2. Crea un índice único en tracking_id (no puede haber dos paquetes con el mismo).
#   3. Inserta paquetes de ejemplo si la colección está vacía.

import sys
from datetime import datetime, timedelta, timezone

from database import ESTADOS, bd, paquetes

# ---------- 1. Reglas de validación ($jsonSchema) ----------
# MongoDB rechaza cualquier documento que no cumpla estas reglas.
VALIDACION = {
    "$jsonSchema": {
        "bsonType": "object",
        "required": ["tracking_id", "estado", "remitente", "destinatario",
                     "descripcion", "peso_kg", "historial_eventos", "total_eventos"],
        "properties": {
            "tracking_id": {"bsonType": "string", "description": "Código de rastreo"},
            "estado": {"enum": ESTADOS, "description": "Solo se aceptan los estados
definidos"},
            "remitente": {  # subdocumento
                "bsonType": "object",
                "required": ["nombre", "ciudad"],
            },
            "destinatario": {  # subdocumento
                "bsonType": "object",
                "required": ["nombre", "direccion", "ciudad"],
            },
            "descripcion": {"bsonType": "string"},
            "peso_kg": {"bsonType": "number", "minimum": 0},
            "repartidor": {"bsonType": ["object", "null"]},  # puede no tener repartidor aún
            "etiquetas": {  # arreglo de textos
                "bsonType": "array",
                "items": {"bsonType": "string"},
            },
            "historial_eventos": {  # arreglo de subdocumentos
                "bsonType": "array",
                "items": {
                    "bsonType": "object",
                    "required": ["fecha", "estado", "ubicacion"],
                    "properties": {
                        "fecha": {"bsonType": "date"},
                        "estado": {"enum": ESTADOS},
                        "ubicacion": {"bsonType": "string"},
                        "comentario": {"bsonType": "string"},
                    },
                },
            },
            "total_eventos": {"bsonType": "number", "minimum": 0},
        },
    }
}




# ... (aquí van las funciones y los 8 paquetes de ejemplo) ...
```

```
# ---------- Ejecución ----------
if __name__ == "__main__":
    # 1. Crear la colección con validación (o actualizar la validación si ya existe)
    if "paquetes" in bd.list_collection_names():
        bd.command("collMod", "paquetes", validator=VALIDACION)
        print("Validación actualizada en la colección 'paquetes'")
    else:
        bd.create_collection("paquetes", validator=VALIDACION)
        print("Colección 'paquetes' creada con validación")


    # 2. Índice único para el código de rastreo
    #    (si ya existía un índice normal en tracking_id, se cambia por uno único)
    indices = paquetes.index_information()
    if "tracking_id_1" in indices and not indices["tracking_id_1"].get("unique"):
        paquetes.drop_index("tracking_id_1")
    paquetes.create_index("tracking_id", unique=True)
    print("Índice único en tracking_id listo")


    # 3. Datos de ejemplo
    if len(sys.argv) > 1 and sys.argv[1] == "reiniciar":
        borrados = paquetes.delete_many({})
        print(f"Se borraron {borrados.deleted_count} paquetes")


    if paquetes.count_documents({}) == 0:
        resultado = paquetes.insert_many(datos_de_ejemplo())
        print(f"Se insertaron {len(resultado.inserted_ids)} paquetes de ejemplo")
    else:
        print("La colección ya tiene datos, no se insertó nada")
```

frontend/app.js (inicio: función pedir, que usa fetch)

```
// ------------------------------------------------------------
// Frontend: habla con la API usando fetch (JSON por HTTP).
// Nunca se recarga la página: solo se vuelve a dibujar la lista.
// ------------------------------------------------------------

const API = "/api";
const ESTADOS = ["Creado", "En tránsito", "En reparto", "Entregado", "Cancelado"];
const COLOR_ESTADO = {
  "Creado": "secondary", "En tránsito": "info", "En reparto": "warning",
  "Entregado": "success", "Cancelado": "danger",
};

// Ventanas (modales) de Bootstrap
const modalPaquete = new bootstrap.Modal("#modal-paquete");
const modalEvento = new bootstrap.Modal("#modal-evento");
const modalHistorial = new bootstrap.Modal("#modal-historial");

let paquetesActuales = [];  // última lista recibida de la API
let idEditando = null;      // null = creando, texto = editando ese _id
let idEvento = null;        // paquete al que se le agrega un evento
let etiquetas = [];         // etiquetas del formulario (arreglo)



// ---------- Utilidades ----------

// Hace una petición a la API y regresa el JSON (o lanza el error que mandó el backend)
```

```
async function pedir(url, metodo = "GET", cuerpo = null) {
  const opciones = { method: metodo, headers: { "Content-Type": "application/json" } };
  if (cuerpo) opciones.body = JSON.stringify(cuerpo);
  const respuesta = await fetch(API + url, opciones);
  const datos = await respuesta.json();
  if (!respuesta.ok) {
    // FastAPI manda el error como texto, o como lista si el JSON no pasó la validación
    const detalle = Array.isArray(datos.detail) ? "Datos inválidos: revisa el formulario" :
datos.detail;
    throw new Error(detalle || "Error en la petición");
  }
  return datos;
}
```

frontend/app.js (consultar con filtros)

```
// ---------- R: listar con filtros ----------
async function cargarPaquetes() {
  // Arma la URL con los filtros: /api/paquetes?buscar=..&estado=..&estado=..&repartidor=..
  const params = new URLSearchParams();
  const buscar = document.getElementById("buscar").value.trim();
  const repartidor = document.getElementById("filtro-repartidor").value;
  if (buscar) params.append("buscar", buscar);
  if (repartidor) params.append("repartidor", repartidor);
  document.querySelectorAll(".filtro-estado:checked").forEach(c => params.append("estado",
c.value));

  paquetesActuales = await pedir(`/paquetes?${params}`);
  dibujarPaquetes();
}
```

frontend/app.js (agregar evento, marcar entregado y eliminar)

```
function abrirEvento(id) {
  idEvento = id;
  formEvento.reset();
  document.getElementById("evento-tracking").textContent = paquetesActuales.find(p => p._id
=== id).tracking_id;
  modalEvento.show();
}

formEvento.addEventListener("submit", async (ev) => {
  ev.preventDefault();
  try {
    await pedir(`/paquetes/${idEvento}/evento`, "PUT", {
      estado: formEvento.estado.value,
      ubicacion: formEvento.ubicacion.value,
      comentario: formEvento.comentario.value,
    });
    modalEvento.hide();
    mensaje(`Evento agregado: ${formEvento.estado.value}`);
    cargarPaquetes();
  } catch (error) {
    mensaje(error.message, "danger");
  }
});

// Atajo: marcar como Entregado en la ciudad del destinatario
```

```
async function marcarEntregado(id) {
  const p = paquetesActuales.find(x => x._id === id);
  try {
    await pedir(`/paquetes/${id}/evento`, "PUT", {
      estado: "Entregado", ubicacion: p.destinatario.ciudad, comentario: "Entregado al
destinatario",
    });
    mensaje(`${p.tracking_id} marcado como Entregado`);
    cargarPaquetes();
  } catch (error) {
    mensaje(error.message, "danger");
  }
}



// ---------- D: eliminar (solo estado Creado) ----------
async function eliminar(id) {
  const p = paquetesActuales.find(x => x._id === id);
  if (!confirm(`¿Eliminar el paquete ${p.tracking_id}?`)) return;
  try {
    await pedir(`/paquetes/${id}`, "DELETE");
    mensaje("Paquete eliminado", "secondary");
    cargarRepartidores();
    cargarPaquetes();
  } catch (error) {
    mensaje(error.message, "danger");
  }
}
```